Enterprise RAG Project – Complete Revision Tutorial

This is not just a project explanation. It is a revision guide for interviews. We will study the project exactly as an LLM Engineer would build and explain it.

Chapter 1 – The Problem Statement

Suppose your company has:

- HR Policies
- Employee Handbook
- Banking SOPs
- Compliance Documents
- Fraud Detection Reports
- Security Policies

An employee asks:

> "How many maternity leave days are allowed?"

A normal LLM like Gemini or ChatGPT cannot reliably answer because:

- It was not trained on your company's private documents.
- Even if it knows a generic answer, it does not know **your company's policy**.
- Uploading every PDF into the prompt is impossible because of context window limitations.

This is the problem RAG solves.

Chapter 2 – What is RAG?

Retrieval-Augmented Generation means:

1. Retrieve only the relevant information.
2. Give that information to the LLM.
3. Let the LLM answer using only that information.

Instead of relying only on the model's memory, we provide external knowledge at runtime.

High-level flow:

User Question ↓ Retriever ↓ Relevant Documents ↓ Prompt ↓ Gemini ↓ Answer

This allows the knowledge base to change without retraining the model.

Chapter 3 – Initial Architecture (Naive RAG)

Our first implementation looked like this:

PDF ↓ Document Loader ↓ Document Objects ↓ Chunking ↓ Embeddings ↓ FAISS ↓ Similarity Search ↓ Prompt Template ↓ Gemini ↓ Answer

This works, but it has several limitations that we improved later.

Chapter 4 – Document Loading

Why do we need a Document Loader?

A PDF is just a file.

An LLM cannot process PDFs directly.

The loader extracts the text and converts it into LangChain `Document` objects.

We used:

PyMuPDFLoader

Why?

- Fast
- Good text extraction
- Reliable for digital PDFs

Output:

List[Document]

Each Document contains:

- page_content
- metadata

Metadata includes:

- source
- page
- author
- title
- creation date
- etc.

Why metadata matters:

- Source citations
- Metadata filtering
- Auditing
- Enterprise traceability

Chapter 5 – Why Document Objects?

Instead of storing plain strings:

"Isolation Forest..."

LangChain stores:

Document(page_content=..., metadata=...)

This allows us to preserve context about where the text came from.

Without Document objects, later features like citations become difficult.

Chapter 6 – Chunking

A single PDF page may contain multiple unrelated topics.

Example:

- Leave Policy
- Dress Code
- Payroll
- Insurance

Creating one embedding for all of them mixes unrelated concepts.

Instead we split the document.

We used:

RecursiveCharacterTextSplitter

Why?

Instead of splitting blindly every 1000 characters, it tries:

Paragraph ↓

Sentence ↓

Word ↓

Character

This preserves semantic meaning.

Configuration:

chunk_size = 1000

chunk_overlap = 200

Why overlap?

Without overlap:

Sentence may be cut in half.

With overlap:

Context is preserved across chunk boundaries.

Chapter 7 – Embeddings

LLMs do not search using text.

They search using vectors.

Example:

Laptop

Notebook Computer

Although the words differ,

their embeddings are close together.

Embedding Model:

all-MiniLM-L6-v2

Why?

- Free

- Fast
- 384 dimensions
- Excellent for semantic search

Output:

Chunk

↓

384-dimensional vector

Chapter 8 – FAISS (Initial Vector Database)

Initially we used FAISS.

Why?

- Lightweight
- In-memory
- Fast
- Good for prototypes

Flow:

Chunks

↓

Embeddings

↓

FAISS Index

↓

Similarity Search

Limitation:

If the notebook restarts, the index is lost.

Chapter 9 – ChromaDB (Production Improvement)

We replaced FAISS with ChromaDB.

Why?

- Persistent storage
- Better metadata handling
- Easier production deployment

Instead of rebuilding embeddings every run, Chroma stores them on disk.

Chapter 10 – Retriever

Initially:

vector_store.similarity_search()

Later:

retriever = vector_store.as_retriever()

Why?

Because our application should depend on a retrieval interface, not a specific database implementation.

This allows us to replace Chroma with Pinecone, Qdrant, Weaviate, etc., without changing business logic.

Chapter 11 – Prompt Engineering

The prompt is the bridge between retrieval and generation.

System message:

Defines the model's role and rules.

Human message:

Contains:

Context

Question

Important rule:

Never allow the model to answer outside the retrieved context.

Otherwise it is no longer behaving as a RAG assistant.

Chapter 12 – Context Formatting

Retriever returns:

List[Document]

Gemini expects:

String

Therefore:

Retriever ↓

List[Document]

↓

format_docs()

↓

String

↓

Prompt

This small helper is essential because LLMs cannot directly consume Python Document objects.

Chapter 13 – Gemini

Input:

Prompt

Output:

AIMessage

We later convert that into plain text before returning it.

Chapter 14 – Source Citations

Initially:

Answer only.

Later:

Answer

+

Source

+

Page Number

We did not ask Gemini to invent citations.

Instead we used document metadata.

This prevents hallucinated citations.

Chapter 15 – Reranker

Retriever uses embedding similarity.

Embedding similarity is approximate.

Reranker performs a deeper comparison between:

Question

+

Chunk

and assigns a relevance score.

Pipeline:

Question

↓

Retriever (Top 10)

↓

Reranker

↓

Top 3

↓

Gemini

Benefits:

- Better retrieval quality
- Less irrelevant context
- Lower token usage
- More accurate answers

Chapter 16 – Final Architecture

User Question

↓

Retriever

↓

Top 10 Documents

↓

Reranker

↓

Top 3 Documents

↓

format_docs()

↓

Prompt Template

↓

Gemini

↓

Answer

↓

Source Citations

↓

Final Response

Chapter 17 – Evolution of the Project

We started with:

Single PDF

↓

FAISS

↓

similarity_search()

↓

Gemini

↓

Answer

We evolved to:

Multiple PDFs

↓

Chunking

↓

ChromaDB

↓

Retriever

↓

Reranker

↓

Prompt Template

↓

Gemini

↓

Source Citations

↓

Enterprise Knowledge Assistant

Every improvement solved a limitation of the previous version.

Chapter 18 – Why These Technologies?

- **PyMuPDFLoader**: Fast and reliable PDF text extraction.
- **RecursiveCharacterTextSplitter**: Preserves semantic boundaries while chunking.
- **all-MiniLM-L6-v2**: Free, lightweight embedding model with good semantic performance.
- **ChromaDB**: Persistent vector database with metadata support.
- **Retriever**: Decouples retrieval logic from the underlying vector store.
- **Gemini**: Generates natural language answers from retrieved context.
- **Prompt Template**: Injects retrieved context and instructions into the model.
- **Metadata**: Enables citations, filtering, and traceability.
- **Reranker**: Improves retrieval quality before generation.

Chapter 19 – What Changed During Development?

- Single PDF → Multiple PDFs
- FAISS → ChromaDB
- similarity_search() → Retriever
- Raw documents → format_docs()
- Answer only → Answer + Source citations
- Basic retrieval → Retrieval + Reranking
- Notebook cells → Modular functions

Each change made the system more maintainable, scalable, or accurate.

Chapter 20 – Complete Request Flow

1. User asks a question.
2. The retriever searches the vector database.
3. Top candidate documents are returned.
4. (Optional) A reranker reorders those candidates.
5. The selected documents are converted into a single context string.
6. The prompt template combines the context with the user's question.
7. Gemini generates an answer using only the supplied context.
8. The application appends source citations using document metadata.
9. The final response is returned to the user.